

MODULE 19 · PART 6 - SETS AND GROUPING

Grouping Sets, Rollup and Cube

Several levels of aggregation in one pass, and the null that means two different things.

PART 6 of 8 MODULE 19 of 26 RUNS about 26 min

Why this closes Part 6

- ◆ **Module eighteen combined results.** Several queries into one, with rules about nulls and duplicates.
- ◆ **This one multiplies them.** One query into several levels, from one scan.
- ◆ **And it replaces the pattern module eighteen ended on,** which is a UNION ALL of the same query with different GROUP BY clauses.
- ◆ **While introducing an ambiguity that union did not have,** because the union version wrote its own literal NULLs deliberately.

THE TAKEAWAY

The clause is shorter, faster and one definition instead of three. It also collapses two kinds of null into one column, which the union version at least did on purpose.

One query, **several GROUP BY clauses at once**, and one scan of the data.

The three forms

```
1 -- name the levels you want
2 GROUP BY GROUPING SETS ((region_key, store_format), (region_key), ())
3
4 -- ROLLUP: a hierarchy, from most detailed to grand total
5 GROUP BY ROLLUP (region_key, store_format)
6     -- equivalent to the GROUPING SETS above
7
8 -- CUBE: every combination
9 GROUP BY CUBE (region_key, store_format)
10     -- adds (store_format) on its own
```

- 5 ROLLUP of n columns gives $n+1$ grouping sets. It assumes the columns form a hierarchy, and drops them right to left.
- 9 CUBE of n columns gives 2 to the power n . It assumes nothing, and asks for every combination.

THE TAKEAWAY

GROUPING SETS is the general form and the other two are abbreviations of it. Anything ROLLUP or CUBE can do, GROUPING SETS can name explicitly.

Reading a grouping set out loud

WRITTEN

`GROUPING SETS ((a,b),(a),())`

`ROLLUP (a, b)`

`CUBE (a, b)`

`GROUPING(b) = 1`

`GROUPING(a, b) = 2`

SAID

group by a and b, and by a, and by nothing

...the same thing, if a contains b

...and also by b on its own

this row is a subtotal across all values of b

a is a real key here, b is a subtotal. Binary 10.

THE TAKEAWAY

The empty parentheses are the grand total, and they are easy to miss. Leaving them out gives you subtotals with nothing to total against.

Measured, on 7.07 million rows

CLAUSE	ROWS OUT
GROUP BY region_key, store_format	21
GROUP BY ROLLUP (region_key, store_format)	29
GROUP BY GROUPING SETS ((r,f),(r),())	29
GROUP BY CUBE (region_key, store_format)	32
GROUP BY CUBE (r, f, on_promo)	96

THE TAKEAWAY

21 detail rows, plus 7 regional subtotals, plus 1 grand total is 29. CUBE adds the 3 format subtotals to make 32, and a third dimension triples it again.

ROLLUP or CUBE

- ◆ **ROLLUP assumes a hierarchy.** Region contains format, so region alone is meaningful and format alone is not asked for.
- ◆ **CUBE assumes nothing.** Every combination, including format across all regions.
- ◆ **ROLLUP is $n+1$ grouping sets. CUBE is 2^n .** Three columns is four sets against eight.
- ◆ **And GROUPING SETS is neither.** Name the four levels the report actually shows, and compute no others.

THE TAKEAWAY

Year, quarter, month is a hierarchy and wants ROLLUP. Region, format, channel is not, and wants either CUBE or a named list of the levels somebody will look at.

Six rows, and every level by hand

```

1 region format amount
2   1   Conv   100
3   1   Super  150
4   2   Conv   50
5   2   Super   1
6
7 ROLLUP(region, format) gives 7 rows:
8   (1,Conv,100) (1,Super,150) (2,Conv,50) (2,Super,1) detail
9   (1,NULL,250) (2,NULL,51)      region
10  (NULL,NULL,301)              grand

```

10 Sum every amount column in the output: $301 + 301 + 301 = 903$. Three times the truth, from four rows of data.

THE TAKEAWAY

Four rows of data and seven rows of output. Summing the output column gives 903 against a real total of 301, which is the non-additivity in four rows.

Testing a subtotalled result

- ◆ **A nullable grouping column with real nulls in it**, which is the only way to see the two kinds of null collide.
- ◆ **Two values in each dimension**, so a subtotal is genuinely the sum of more than one row.
- ◆ **A combination that does not exist**, so you can see whether it appears as a row or is absent.
- ◆ **And the grand total checked against the detail rows only**, which is the assertion that catches the non-additivity.

THE TAKEAWAY

Six rows of fixture. The first point is the one nobody includes, because test data is usually tidy and production data is 62% null.

Subtotal rows are marked with **NULL**. Your data already uses NULL for something else.

Two rows, both saying NULL

```
1 GROUP BY ROLLUP (region_key, promo_key)
2
3 region  promo  count      GROUPING(promo)
4      1   NULL  628,150          0      <- no promotion
5      1   NULL  628,317          1      <- all promotions
6
7 -- 167 rows apart. Same region. Same column reading NULL.
8 -- One is a real group. One is a subtotal.
```

- 4 628,150 sales in region 1 that carried no promotion. A genuine group, whose key happens to be null.
- 5 628,317 sales in region 1 across all promotions. A subtotal, marked by a null the clause invented.

THE TAKEAWAY

Measured on the capstone database. Two adjacent rows, 167 apart, indistinguishable by eye, and only the GROUPING column separates them.

Why this is worse than it looks

- ◆ **The two rows are nearly equal**, because 62% of lines have no promotion. They look like a rounding artefact or a duplicate.
- ◆ **A reader who deduplicates loses one of them**, and which one they lose depends on which they saw first.
- ◆ **A reader who sums the column double counts**, because one of those rows is already inside the other.
- ◆ **And nothing in the output says so**. The distinction exists only in a function you have to have called.

THE TAKEAWAY

If the two figures were wildly different somebody would investigate. They are 0.03% apart, which is exactly the range where nobody does.

GROUPING, and what it returns

```
1 SELECT region_key, promo_key, COUNT(*) AS n,  
2        GROUPING(region_key)           AS g_region,  
3        GROUPING(promo_key)            AS g_promo,  
4        GROUPING(region_key, promo_key) AS grouping_id  
5 FROM base  
6 GROUP BY ROLLUP (region_key, promo_key);  
7  
8 -- GROUPING(col) is 1 if this row is a subtotal over that column,  
9 -- and 0 if the column is a real grouping key on this row.
```

4 GROUPING of several columns returns a bitmask, so two columns give 0, 1, 2 or 3. One number identifies the level.

THE TAKEAWAY

GROUPING answers exactly one question: is this null mine, or yours? Everything else about disambiguating a subtotal is built on that one bit.

Labelling the rows

```
1 SELECT
2     CASE WHEN GROUPING(region_key) = 1 THEN 'All regions'
3         ELSE region_key::VARCHAR END AS region,
4     CASE WHEN GROUPING(promo_key) = 1 THEN 'All promotions'
5         WHEN promo_key IS NULL THEN 'No promotion'
6         ELSE promo_key::VARCHAR END AS promotion,
7     GROUPING(region_key, promo_key) AS level,
8     COUNT(*) AS n
9 FROM base GROUP BY ROLLUP (region_key, promo_key);
```

5 The order of the branches is the whole thing. Test GROUPING first, because the null test would catch both.

THE TAKEAWAY

Two rows that both read NULL now read 'All promotions' and 'No promotion'. The distinction is in the output rather than in a function somebody forgot to call.

A promotional performance report groups by region and promotion with ROLLUP. Commercial notice two rows per region that both show a blank promotion, with counts differing by around 0.03%.

They assume the pipeline is emitting duplicates and ask for it to be deduplicated. promo_key is null on 62% of sales lines.

What is actually happening?

- A One row is the 'no promotion' group and the other is the subtotal across all promotions. GROUPING tells them apart.
- B ROLLUP is emitting duplicate rows for each region, which is a known behaviour when a grouping column is nullable.
- C The two counts come from different snapshots of the fact table, so the small difference is a load timing artefact.
- D ROLLUP should be CUBE, which would produce a single unambiguous row per region instead of two.

◆ PAUSE AND DECIDE

What is actually happening?

- A

 One is a group, one is a subtotal
628,150 sales with no promotion, and 628,317 across all promotions. Both are correct and they are different facts.
- B

 ROLLUP emits duplicates
It does not. Every grouping set produces distinct rows; these two come from different sets and happen to look alike.
- C

 Different snapshots
Both rows come from one scan of one table in one query. There is no second snapshot to disagree with.
- D

 Use CUBE
Adds grouping sets rather than removing them. The same ambiguity appears, on more rows.

THE SIGNAL

"Differing by around 0.03%." Two figures that close look like an error. They are the difference between sales with no promotion and sales across all promotions, which on this data is almost the same number.

A subtotal row is **a different kind of row**. It is not more data, it is a summary sitting in the same table.

Ordering a subtalled report

```
1 SELECT region_key, store_format, SUM(net_sales) AS sales,  
2        GROUPING(region_key, store_format) AS lvl  
3 FROM base  
4 GROUP BY ROLLUP (region_key, store_format)  
5 ORDER BY GROUPING(region_key),      -- regions first, grand total last  
6        region_key,  
7        GROUPING(store_format),      -- detail before each subtotal  
8        store_format;
```

7 Order by the GROUPING value, not by the column. Ordering by the column puts the subtotal wherever NULL sorts.

THE TAKEAWAY

Without this, the subtotal for region 1 sorts wherever NULL sorts in store_format, which on Postgres and Redshift is last and on SQL Server is first.

Where the subtotal rows should sit

- ◆ **Detail rows first, then their subtotal**, which is how a person reads a table and how a spreadsheet lays one out.
- ◆ **Which means ordering by GROUPING ascending**, because zero is detail and one is subtotal.
- ◆ **Ordering by the nullable column instead is engine-dependent**, and puts the subtotal at the wrong end on half of them.
- ◆ **And the grand total is always last**, which falls out of ordering by the outermost GROUPING first.

THE TAKEAWAY

Module two's null-ordering problem, arriving in a report layout. The fix is the same: sort on something you computed rather than on a null's position.

A result containing subtotals is **not additive**. Summing its column counts everything more than once.

Why it triples

```
1 ROLLUP (region_key, store_format) over 7,069,736 rows
2
3 21 detail rows      every sale, once
4 + 7 region subtotals every sale, again
5 + 1 grand total     every sale, a third time
6 ---
7 29 rows, and SUM over all of them is 3x the truth
8
9 -- filter to GROUPING(...) = 0 for the detail level only
```

7 Not an approximation. Exactly three times, because every sale appears in exactly three of the 29 rows.

THE TAKEAWAY

A rolled-up result is correct at every level and must never be aggregated across levels. Module nine said the same about hierarchy totals, and it is the same arithmetic.

Who gets caught by that

- ◆ **A reporting tool that totals a column by default**, which most of them do, and which nobody switches off.
- ◆ **A downstream model that reads the table**, sees rows and measures, and aggregates them as it would any other.
- ◆ **An export to a spreadsheet**, where somebody drags the column into a sum without knowing what the rows are.
- ◆ **So carry the level as a column**, and make every consumer filter on it. It is one integer, and it prevents all three.

THE TAKEAWAY

The GROUPING bitmask is the level. Carrying it means a consumer can filter to the detail rows without understanding ROLLUP at all.

A rolled-up regional sales table is published for reporting. A dashboard built on it shows a company total of exactly three times the figure in the finance pack.

Spot checks confirm every individual row in the table is correct, including the subtotals and the grand total. The dashboard sums the `net_sales` column with no filter.

What is the cause?

- A The source fact table contains duplicate rows, which the rollup is faithfully reproducing at every level.
- B ROLLUP is producing more levels than requested, so extra subtotals are inflating the total.
- C A rolled-up result is not additive. Every sale appears in a detail row, a subtotal and the grand total.
- D The dashboard should use CUBE, which produces mutually exclusive rows that can safely be summed.

◆ PAUSE AND DECIDE

What is the cause?

A Duplicates at source

Would inflate every level equally and the factor would not be exactly three. Spot checks also confirm the rows are correct.

C Not additive

Three grouping sets means every sale is counted three times. Filter on the GROUPING bitmask to get the detail level only.

B Extra levels

ROLLUP of two columns produces exactly three grouping sets. Three is the number of levels, and it is also the multiplier.

D Use CUBE

CUBE adds a fourth grouping set, so the total would be four times rather than three. It makes the symptom worse.

THE SIGNAL

"Exactly three times." An exact small integer multiple is the signature of a result being counted once per level. Three grouping sets, three counts, three times.

Percent of parent

```
1 SELECT region_key, store_format, sales,  
2         sales / NULLIF(MAX(CASE WHEN lvl = 1 THEN sales END)  
3                           OVER (PARTITION BY region_key), 0) AS pct_of_region  
4 FROM rolled_up  
5 WHERE lvl IN (0, 1);
```

3 The subtotal row is already in the result, so the denominator is a window over it rather than a second aggregation.

THE TAKEAWAY

This is the one genuine advantage of having subtotals in the same table. The denominator is a row you already have, so percent-of-parent costs one window.

CUBE of n columns is **2 to the n** grouping sets. Six dimensions is sixty four.

How the rows multiply

```
1 CUBE (region_key, store_format)      -> 32 rows
2 CUBE (region_key, store_format, promo) -> 96 rows
3
4 -- measured. Each dimension roughly triples it here,
5 -- because it doubles the grouping sets AND adds levels
6 -- whose cardinality is the product of the others.
7
8 -- CUBE over six dimensions with 10 values each:
9 -- 64 grouping sets, and up to 11^6 = 1,771,561 rows
```

9 Eleven rather than ten per dimension, because each one also contributes its 'all values' level.

THE TAKEAWAY

The row count of a full CUBE is the product of one-plus-cardinality across every dimension. That is a multiplication, and it gets away from people quickly.

Why GROUPING SETS beats CUBE in practice

- ◆ **A report shows a handful of levels**, not every combination. Nobody looks at format across all regions and all channels.
- ◆ **So name the levels**. Four grouping sets instead of sixty four, and the query says which four somebody asked for.
- ◆ **CUBE computes levels nobody will read**, and then somebody has to filter them out downstream anyway.
- ◆ **And the named list is documentation**. A reviewer can see exactly which aggregations this report contains.

THE TAKEAWAY

CUBE is right when the consumer genuinely explores every combination, which means an OLAP tool rather than a report. For a report, the levels are known in advance.

An analyst wants subtotals across six dimensions for a commercial review: region, format, channel, category, promotion type and period. They propose CUBE over all six.

The review deck contains four tables: by region and format, by category, by channel and period, and a grand total. The fact table has 7 million rows.

What should they be told?

- A CUBE is the right tool for multi-dimensional subtotals. Run it and let the reporting layer filter to what the deck shows.
- B CUBE of six columns is 64 grouping sets. Name the four levels the deck shows with GROUPING SETS instead.
- C Use ROLLUP over the six columns, which produces fewer grouping sets and is therefore the safer choice.
- D Split it into six separate queries, one per dimension, and union the results together.

◆ PAUSE AND DECIDE

What should they be told?

- A

 Run CUBE and filter later
Computes 64 levels to display 4, then discards 60. The filtering happens after the expensive part, which is the wrong order.
- B

 Name the four levels
Four grouping sets instead of 64, and the query documents which aggregations the deck contains.
- C

 Use ROLLUP
Seven grouping sets rather than 64, and they are the wrong seven. ROLLUP drops columns right to left; these six are not a hierarchy.
- D

 Six separate queries
Six scans to produce four tables, and it is module eighteen's union-of-queries pattern that this clause exists to replace.

THE SIGNAL

"The deck contains four tables." The levels are known in advance, written down, and there are four of them. CUBE is for exploration, and a deck is not exploration.

Windows over a subtalled result

- ◆ **They work, and they see the subtotal rows**, which is useful for percent-of-parent and dangerous for everything else.
- ◆ **A RANK over a rolled-up result ranks subtotals alongside detail**, and the grand total wins every time.
- ◆ **So filter to one level before ranking**, or partition by the GROUPING value so each level ranks within itself.
- ◆ **Module three's question, one more time.** Rank within what, and does the population include the summaries?

THE TAKEAWAY

A top-ten report over a rolled-up table puts the grand total at number one. It is not wrong, and nobody asked for it.

Distinct counts do not roll up

- ◆ **The distinct customers across two regions is not the sum of each region's.** A customer shopping in both is counted once.
- ◆ **GROUPING SETS computes each level from the detail**, so the subtotal is correct. It just is not the sum of the rows above it.
- ◆ **Which breaks the subtotal assertion**, because for a distinct count the subtotal legitimately differs from the sum of its children.
- ◆ **Module nine said the same about medians.** Non-additive measures are non-additive at every level.

THE TAKEAWAY

Sums and counts roll up. Distinct counts, medians and percentiles do not, and a subtotalled result containing one cannot be checked by adding up its children.

Support, by engine

ENGINE	WHAT IT HAS
PostgreSQL 9.5+, Oracle, SQL Server	All three, plus GROUPING and GROUPING_ID.
Amazon Redshift	All three, since February 2023. Before that, UNION ALL.
Snowflake, BigQuery, DuckDB	All three.
MySQL 8.0	WITH ROLLUP only. No CUBE, no GROUPING SETS.
MySQL 8.0.1+	GROUPING() exists, and works with WITH ROLLUP.

THE TAKEAWAY

Redshift gained these in February 2023 and QUALIFY in July 2023. Two clauses this course relies on arrived within five months of each other.

MySQL's different spelling

```
1 -- everywhere else: a function in the GROUP BY
2 GROUP BY ROLLUP (region_key, store_format)
3
4 -- MySQL: a modifier after the GROUP BY
5 GROUP BY region_key, store_format WITH ROLLUP
6
7 -- and for anything CUBE or GROUPING SETS would do,
8 -- MySQL needs a UNION ALL per level. Six dimensions
9 -- as a full cube would be 64 branches.
```

- 5 Same semantics, different position. A port in either direction is a one-line edit and will not be caught by a syntax check elsewhere.

THE TAKEAWAY

MySQL has the common case and none of the general one. A report using GROUPING SETS does not port to it, and a ROLLUP report ports with one edit.

A subtotalled operational report using GROUPING SETS is being ported to a MySQL 8.0 instance that backs a store-manager app. The report shows four levels, two of which are not hierarchical.

The engineer doing the port reports that the query is rejected. They ask whether MySQL needs a version upgrade or a different GROUP BY syntax.

What should they be told?

- ☐ A MySQL has supported GROUPING SETS since 8.0. The syntax must be written with the function inside the GROUP BY.
- ☐ B Use CUBE instead, which MySQL does support and which is a superset of any GROUPING SETS specification.
- ☐ C Rewrite it as ROLLUP, since any set of grouping levels can be expressed as a hierarchy in the right column order.
- ☐ D MySQL has WITH ROLLUP only. Express the four levels as a UNION ALL, and use GROUPING() to mark the subtotals.

◆ PAUSE AND DECIDE

What should they be told?

A MySQL has GROUPING SETS

It does not, in any 8.0 release. The rejection is correct behaviour rather than a syntax error to be corrected.

C Rewrite as ROLLUP

ROLLUP only produces prefixes of the column list. Two of these levels are not hierarchical, so no ordering reaches them.

B Use CUBE

MySQL has no CUBE either. And CUBE is not a drop-in for GROUPING SETS; it computes every combination rather than named ones.

D UNION ALL, plus GROUPING

One branch per level, which is module eighteen's pattern. GROUPING has existed since 8.0.1 and still marks the subtotals.

THE SIGNAL

"Two of which are not hierarchical." That detail rules out ROLLUP specifically, and it is the one piece of information that decides between C and D.

The capstone's regional summary

```
1 SELECT
2     COALESCE(region_key::VARCHAR, 'ALL') AS region,
3     COALESCE(store_format, 'ALL FORMATS') AS store_format,
4     GROUPING(region_key) = 1           AS is_national_total,
5     GROUPING(store_format) = 1         AS is_region_subtotal,
6     SUM(net_sales)                     AS net_sales
7 FROM store_day_with_format
8 GROUP BY GROUPING SETS ((region_key, store_format),
9                        (region_key),
10                       ());
```

5 The flags computed before the COALESCE would hide them. This is what the capstone harness checks for.

THE TAKEAWAY

Deliverable d08. Its harness asserts that region subtotals equal the sum of their formats, and that the national total equals the sum of the subtotals rather than of every row.

The subtotal assertion

```
1 SELECT
2     -- every region subtotal equals the sum of its formats
3     COUNT(*) FILTER (WHERE ABS(sub - parts) > 0.01) AS bad_subtotals
4 FROM (
5     SELECT r.region_key, r.net_sales AS sub,
6            SUM(d.net_sales)          AS parts
7     FROM result r JOIN result d
8          ON d.region_key = r.region_key AND d.lvl = 0
9     WHERE r.lvl = 1
10    GROUP BY r.region_key, r.net_sales);
```

9 Join the subtotal rows to the detail rows of the same region, using the level column to tell them apart.

THE TAKEAWAY

The level column is what makes this writable at all. Without it there is no way to join a subtotal to its own detail rows.

What it costs

- ◆ **One scan, rather than one per level.** That is the headline argument, and it is real.
- ◆ **Though modern optimisers rewrite the UNION version well.** Measured here: 0.09 seconds against 0.11, which is not much.
- ◆ **So the real argument is correctness,** not speed. One clause, one definition, and no chance of three branches disagreeing.
- ◆ **And the output is guaranteed consistent,** because all the levels come from the same scan of the same rows.

THE TAKEAWAY

Measured on 7.07 million rows. The clause is faster and not dramatically so, and it is dramatically less likely to be wrong.

Three assertions for a subtotaled result

- ◆ **Each subtotal equals the sum of its detail rows.** One join of the result to itself on the grouping level.
- ◆ **The grand total equals the sum of the detail rows only,** not of every row. Filter on GROUPING first.
- ◆ **Every level you named is present,** with the row count you expect, which catches a grouping set that produced nothing.
- ◆ **The capstone's regional summary checks the first two,** and its harness fails a result where they disagree.

THE TAKEAWAY

The first assertion is what proves the subtotals are subtotals rather than something that happens to sit in the same column.

When not to use these at all

- ◆ **When the consumer is a reporting tool**, which computes its own subtotals and will double count yours.
- ◆ **When the output feeds another model**, because everything downstream now has to filter on a level column.
- ◆ **Use them at the edge**, for an export or a fixed report, which is the same rule as module sixteen's pivot.
- ◆ **And that is not a coincidence.** Both produce a shape for a person, and both stop being a clean table when they do.

THE TAKEAWAY

Pivoted and subtalled results are the two shapes that are correct for a page and wrong for a pipeline. Part 5 and Part 6 arrive at the same rule.

What Part 6 gave you

MODULE	THE ONE THING TO KEEP
18 Set operations	NULL equals NULL here, which makes EXCEPT the right diff tool.
18 Set operations	Columns match by position, and UNION is a deduplication you pay for.
19 Grouping sets	One scan, several levels, and one definition instead of three.
19 Grouping sets	A subtotal NULL and a real NULL share a column. GROUPING separates them.

THE TAKEAWAY

Two modules, four facts. Three of them are about NULL, which is the thread running through this entire course.

Combining the two

```
1  -- module 18's diff, applied to module 19's output
2  SELECT region, store_format, is_region_subtotal, net_sales
3      FROM summary_v1
4  EXCEPT
5  SELECT region, store_format, is_region_subtotal, net_sales
6      FROM summary_v2;
```

- 4 Include the level flags in the comparison, or a detail row and a subtotal row with the same value look identical.

THE TAKEAWAY

The two modules meet here. A subtotalled result is diffed with EXCEPT, and the level flags have to be in the comparison or the diff is ambiguous.

Three things to carry forward

- ◆ **GROUPING is not optional on a nullable column.** It is the difference between a report and a trap.
- ◆ **Carry the level as a column,** because it makes the result readable, filterable and self-checkable at once.
- ◆ **And never sum across levels,** which is the same rule as module nine's hierarchy totals and module seventeen's round trip.

THE TAKEAWAY

Three rules, one idea underneath: a result containing summaries is not a table of facts, and it has to say which rows are which.

Deciding, in four questions

01

Which levels

Write them down. If you can list them, use GROUPING SETS and name them.

02

Hierarchy

Do the columns nest? If so ROLLUP is a shorthand. If not, it produces the wrong levels.

03

Nullable

Is any grouping column nullable? If so, GROUPING is compulsory, not optional.

04

Consumer

Who reads it, and will they sum a column? Carry the level so they can filter.

THE TAKEAWAY

Question three is the one that produces a wrong published number. The other three produce an inefficient query or a wrong shape, both of which somebody notices.

The confusions, settled

SYMPTOM	THE CAUSE
Two near-identical rows with a blank key	A real null and a subtotal null. Use GROUPING.
The total is exactly 3x	Three grouping sets. The result is not additive.
A CUBE returns far more rows than expected	2 to the n grouping sets, times the cardinalities.
ROLLUP gives the wrong subtotals	It drops columns right to left. Your columns are not a hierarchy.
The query is rejected on MySQL	WITH ROLLUP only. No CUBE and no GROUPING SETS.
A subtotal does not match its details	Something filtered between the levels, or the grain is wrong.

THE TAKEAWAY

The first two are the expensive ones, because both produce plausible numbers. The other four fail loudly enough that somebody investigates.

Five habits

- ◆ **Name the levels with GROUPING SETS** rather than reaching for CUBE, because a report's levels are known in advance.
- ◆ **Call GROUPING on every nullable grouping column**, and label the rows, so a subtotal never reads as a real value.
- ◆ **Carry the grouping bitmask as a column**, so a consumer can filter to one level without understanding the clause.
- ◆ **Never sum across levels**, and assert that each subtotal equals the sum of its details.
- ◆ **And check the engine before writing it**. Redshift gained these in 2023, and MySQL still has only ROLLUP.

THE TAKEAWAY

Habits two and three are the same idea at two scales: make the level visible in the output, rather than leaving it implicit in a null.

Part 7 stops asking what a query returns and starts asking how you know it is right.